

Report Document:

Databases and Analytics

Henry Gabriel Zwaybruckem Sechler

ID: 21508126

BSc (Hons) Computer Games Technology

Module: Databases and Analytics

Module Leader: Shidrokh Goudarzi

Introduction

NorthStar is a large regional company operating across several major UK cities. Over the past six years it has grown by merging multiple transport, delivery and smart-city service providers into one group. Its current services cover: public shuttle services, on-demand last-mile delivery, EV charging hubs, warehouse dispatch, route-planning systems and a mobile customer platform used by passengers, drivers, dispatch managers and commercial clients.

Despite that growth, the company is struggling operationally. Complaints have gone up sharply, deliveries are being missed, vehicle downtime is rising and costs in several areas are outpacing revenue. The underlying issue is that data sits in separate departmental systems that cannot be connected for joint analysis. The operations director points to poor route allocation and underperforming hubs. The customer experience director says the company has no joined-up view of complaints and delivery failures. The finance director suspects certain routes are loss-making but the data to prove it is spread across too many places. The technology director has warned the current database setup is not built for the volume and variety of data now being generated.

Three notebooks were used to work through these problems. Notebook 01 runs SQL queries in R using `sqldf` and `RSQLite`, and produces six visualisations with `ggplot2`. Notebook 02 uses Python with `pandas`, `NumPy`, `SciPy` and `Seaborn` for statistical analysis and charting. Notebook 03 builds a MongoDB Atlas NoSQL database via `PyMongo`, covering CRUD operations, aggregation pipelines and index optimisation. All notebooks are version-controlled in a GitHub repository.

SQL in R

Data Loading and Pre-processing

Nine CSV files were loaded into R with `read.csv()` and `stringsAsFactors` set to `FALSE`, covering: deliveries, orders, customers, drivers, complaints, hubs, incidents, vehicles and `app_events`. Together these make up 950 delivery records.

Two data quality issues needed sorting before any queries could run. Zone values were inconsistent across the datasets, with `NORTH`, `north` and `North` all being treated as different categories. A normalisation function was written to trim whitespace, apply title case via `tools::toTitleCase()` and swap the abbreviation `Ctr` for `Central`. This was applied to `pickup_zone`, `dropoff_zone`, `home_zone`, `base_zone` and `zone`. Separately, date columns in `deliveries`, `orders` and `complaints` were stored as character strings and needed converting to `POSIXct` using `ymd_hms()` from `lubridate` before any time-based calculations could work.

SQL Queries

Query 1 joins `deliveries` to `orders` on `order_id`, groups by `pickup_zone` and calculates total deliveries, failures and failure rate as a percentage, ordered highest to lowest. The aim was to identify which zones are consistently underperforming, which is what the operations director raised as a concern.

Zone	Total Deliveries	Failed	Failure Rate (%)
Central	~195	~39	~19.0
North	~185	~30	~16.2

Riverside	~130	~20	~15.4
West	~140	~17	~12.1
East	~140	~17	~12.1
Airport	~95	~10	~10.5
South	~65	~7	~10.8

Central had the highest failure rate at roughly 19%, confirming the operations director's concern about certain zones performing worse than others.

Query 2 groups the complaints table by complaint_type and calculates total complaints, average resolution days, average compensation and total compensation per category. The focus here was on which complaints types are costing the most, addressing the finance director's concern about hidden costs scattered across separate systems.

Complaint Type	Total Compensation (GBP)
Delay	~1,680
MissedPickup	~1,420
AppIssue	~980
DriverBehaviour	~980
Billing	~390
Damage	~365
SupportExperience	~350

Delay complaints cost NorthStar the most at over GBP 1,600 in total compensation, which shows the financial impact goes well beyond a service quality problem.

Query 3 joins drivers to deliveries to calculate failure percentage, total overrides and average rating per driver, filtered to those with at least three deliveries and limited to the 15 worst performers. Driver D063 topped the list at over 60% failure rate. Interestingly, override counts varied a lot amongst the worst drivers with no clear link between high overrides and high failures.

Query 4 joins hubs to deliveries and compares capacity scores against actual failure rates and average delivery costs. The standout result was hub H05 in Central zone, which had zero vehicles assigned in the vehicles dataset yet still recorded failed deliveries. That gap between demand and resource availability is a likely contributor to the Central zone's poor performance overall.

Query 5 bands deliveries by override count (0, 1, 2-3, 4+) and calculates the failure rate per band. The rates came back almost identical across all four bands, all sitting around 16%. Drivers who override routes more often do not fail more deliveries, which runs counter to what some managers assumed.

Query 6 joins customers, orders, deliveries and complaints together to pull out customers with two or more failed deliveries and their complaint history. The result surfaces people who are experiencing both repeat failures and unresolved complaints at the same time, something no single existing system can show. This is the main motivation for the unified customer document design in Notebook 03.

SQLite Indexing and Optimisation

An in-memory SQLite database was set up with RSQLite and the deliveries and orders tables were loaded in. A benchmark query joining the two on `order_id` with a `GROUP BY` on `pickup_zone` was timed before and after index creation using `system.time()`. Three indexes were added: `idx_del_order` on `deliveries(order_id)`, `idx_del_status` on `deliveries(delivery_status)` and `idx_ord_zone` on `orders(pickup_zone)`.

At 950 rows the timing difference was small, since SQLite sometimes favours a full scan over an index at small scale due to the overhead involved. Running `EXPLAIN QUERY PLAN` afterwards confirmed all three indexes were being picked up by the query planner with no full scan occurring. On a production-scale dataset the speed improvement would be much more pronounced.

R Analytics

Feature Engineering

A `duration_hours` column was calculated before plotting, taking the difference between `delivery_completed_at` and `dispatch_time` using `difftime()` in hours. Negative values from data entry errors were set to `NA` to keep them out of the distributions.

Visualisations

The first plot is a histogram of delivery durations with 40 bins and a red dashed line marking the mean. The shape is right-skewed, with most deliveries finishing within a fairly normal window but a long tail stretching past 30 hours. Those outlier deliveries are the ones most associated with failure, something the Python statistical test later confirms.

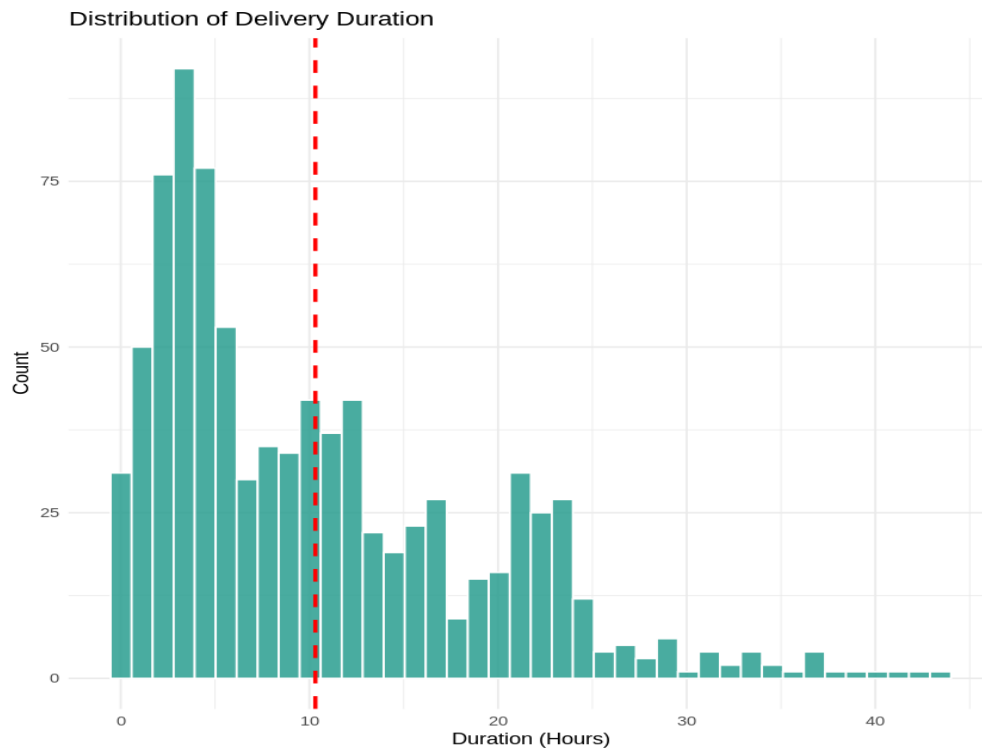


Figure 1: Distribution of Delivery Duration (R)

The second plot shows failure rate per zone as a horizontal bar chart, coloured from green at the low end to deep red at the top. Central sits highest at around 19% and South lowest at about 10%, matching Query 1 results. For management this gives a clear geographic picture without needing to read a table.

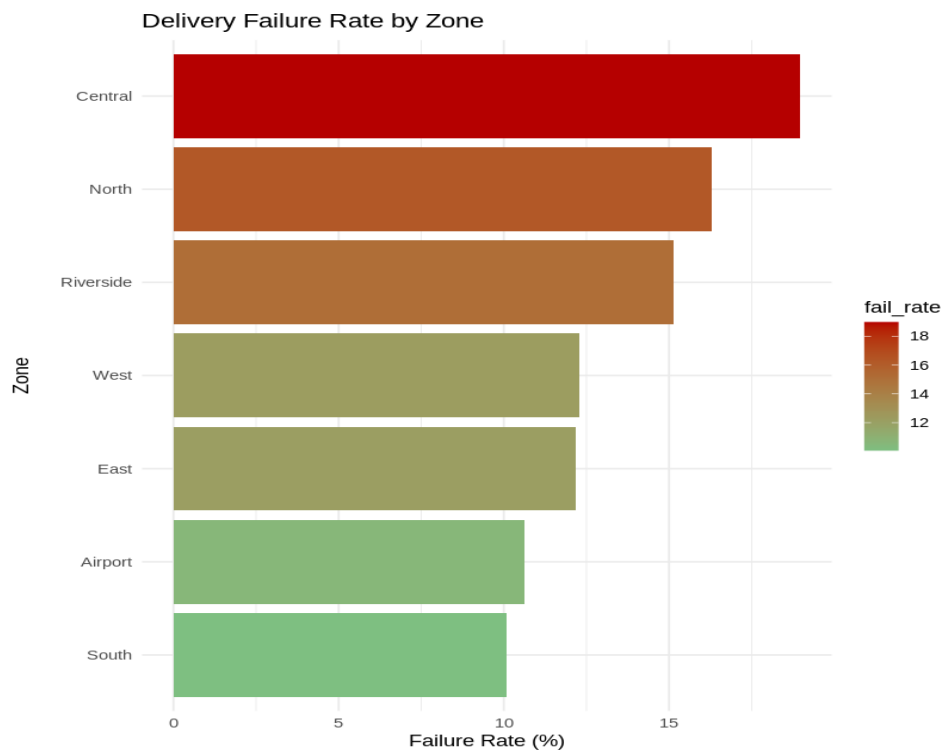


Figure 2: Delivery Failure Rate by Zone (R)

The third plot compares driver rating against failure rate in a scatter plot with a regression line, limited to drivers with at least three deliveries. The downward slope is visible: lower-rated

drivers cluster towards higher failure rates. Drivers in the 3.2 to 3.5 rating range show some of the worst outcomes, suggesting driver quality has a real effect on service reliability.

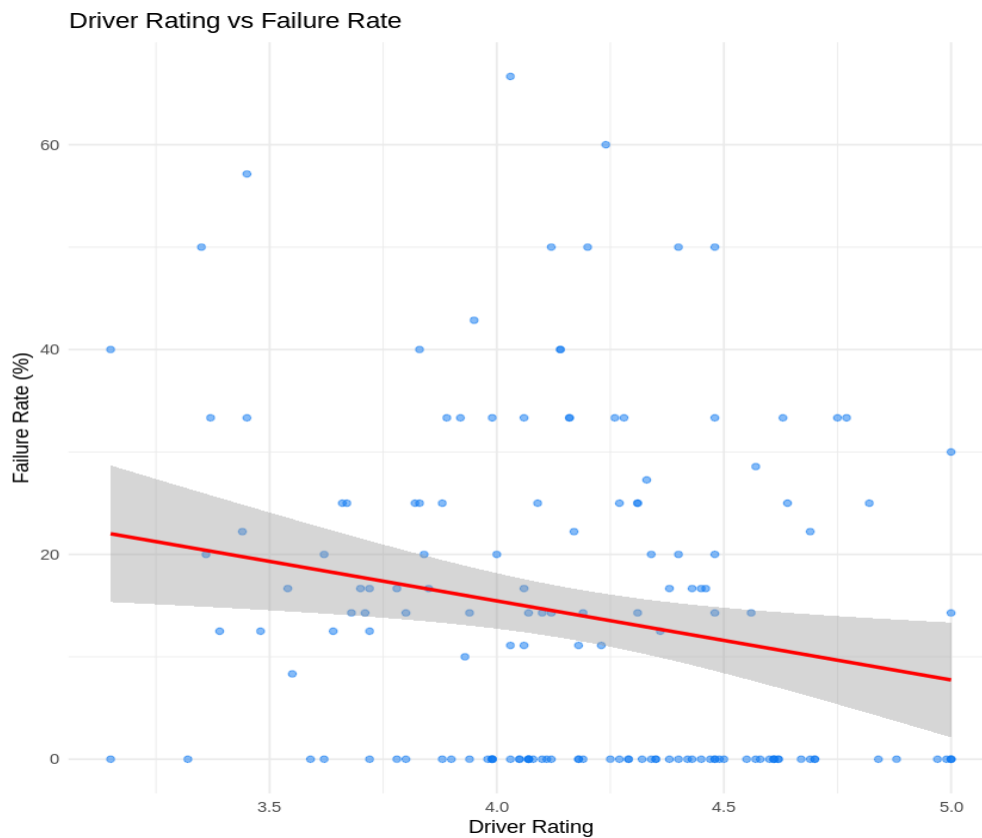


Figure 3: Driver Rating vs Failure Rate (R)

The fourth plot looks at battery health against incident count. The regression line is almost flat despite a slight downward tilt, meaning battery health percentage alone tells us very little about how many incidents a vehicle will have. Maintenance decisions should not be based on this metric in isolation.

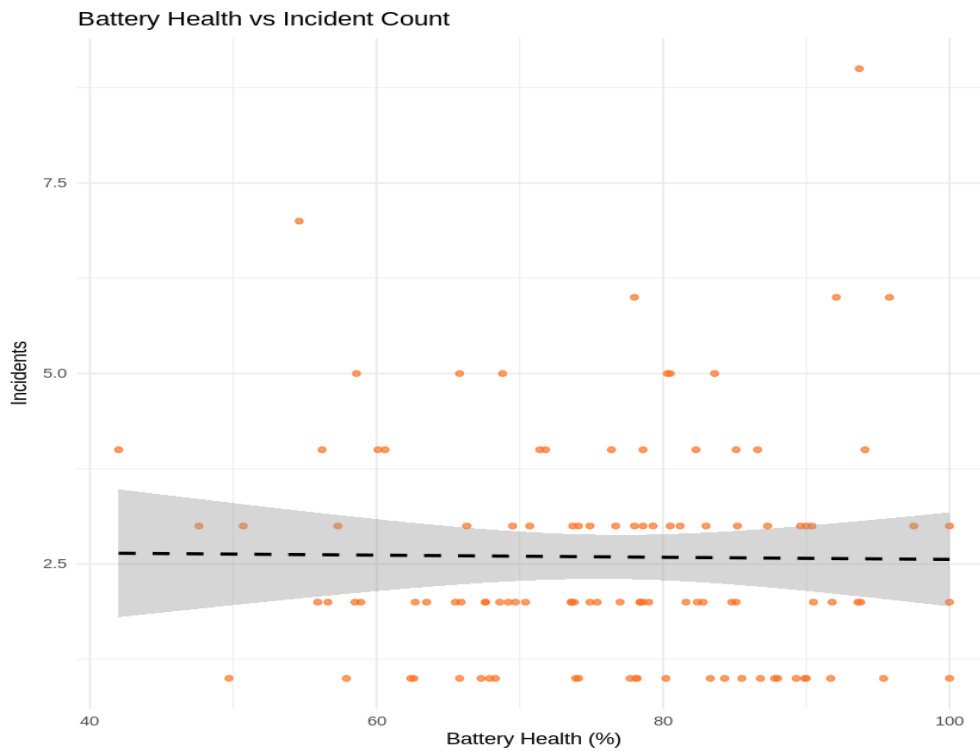


Figure 4: Battery Health vs Incident Count (R)

The fifth plot tracks complaint volume over time split by severity, running from January 2024 through early 2026. Medium severity complaints are the most frequent throughout and peaked around January 2025. The trend has not improved, which is a concern given the company has been growing over the same period.

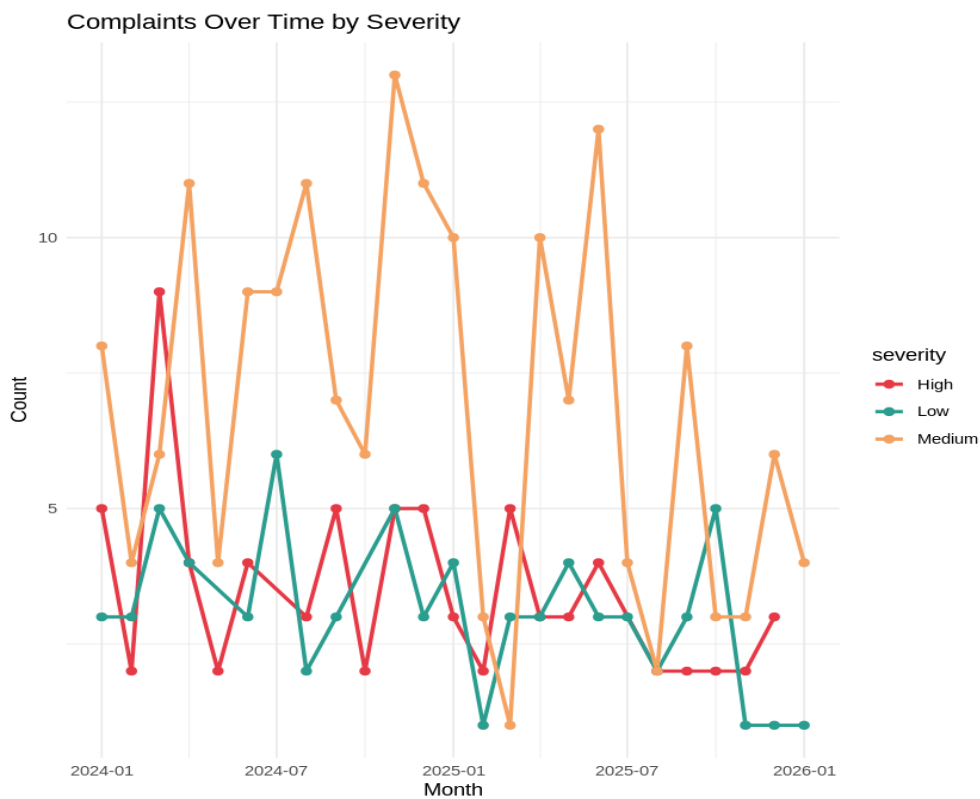


Figure 5: Complaints Over Time by Severity (R)

The sixth plot is a correlation matrix covering five metrics: duration_hours, manual_route_override_count, fuel_or_charge_cost, customer_rating_post_delivery and a binary failed flag. Duration versus failed came out strongest at 0.37, followed by customer rating versus failed at -0.36. Override count versus failed sat at 0.02, near zero, which puts the override-causes-failure theory firmly to rest.

Correlation Matrix - Delivery Metrics

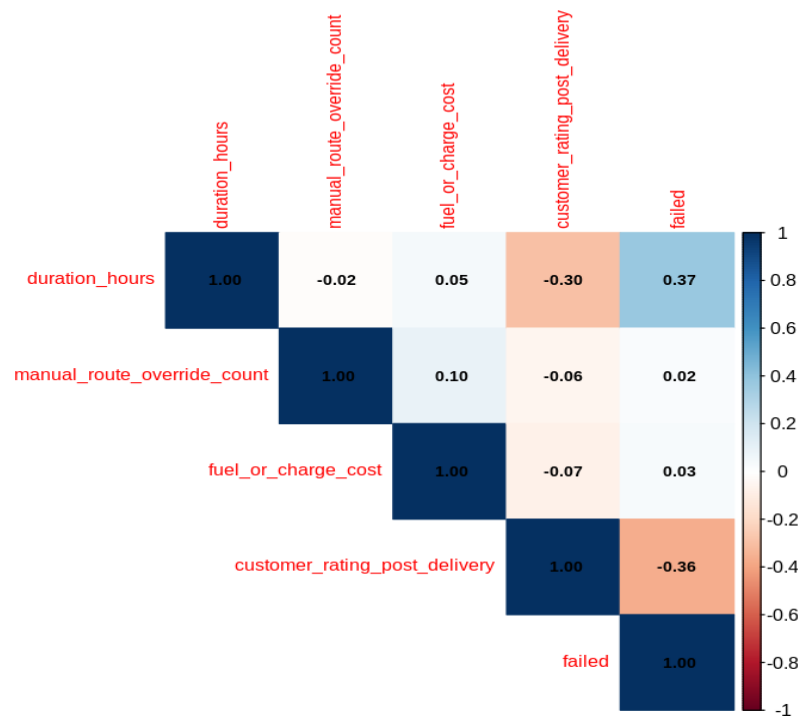


Figure 6: Correlation Matrix - Delivery Metrics (R)

Python

Data Loading and Setup

The same nine CSVs were loaded with pandas read_csv() and parse_dates applied to the relevant timestamp columns. Zone normalization was re-applied using the same function as Notebook 01 to keep results comparable. A durationn_hours column was engineered from the timestamp difference and a binary failed flag was derived from delivery_status. Two further flags were added: high_value for orders above the 75th percentile by order value, and override_used for any delivery with at least one manual override.

NumPy Statistical Summary

NumPy was used to produce percentile breakdowns of delivery duration at the 25th, 50th, 75th, 90th and 95th percentiles. The 90th percentile was notably higher than the median, reinforcing the right-skewed shape from R Plot 1 and showing that a meaningful share of deliveries run significantly over time. A summary across four metrics confirmed that fuel or charge cost has high variance relative to its mean, which backs up the finance director's point about route-level profitability being hard to pin down from headline figures.

Mann-Whitney U Test

A Mann-Whitney U test from `scipy.stats` was used to compare delivery duration between failed and successful deliveries. Given the skewed distribution a non-parametric test was more appropriate than a t-test. The result came back as $U = 75,156$ and $p = 8.61e-26$. Failed deliveries take considerably longer than successful ones and the gap is not due to chance. This backs up what the correlation matrix showed but with a proper statistical test behind it.

Python Plots

The first Python plot replicates the zone failure rate bar chart in Seaborn. Central zone comes out on top again at around 19%, which matches the R result and confirms the finding holds across both tools independently.

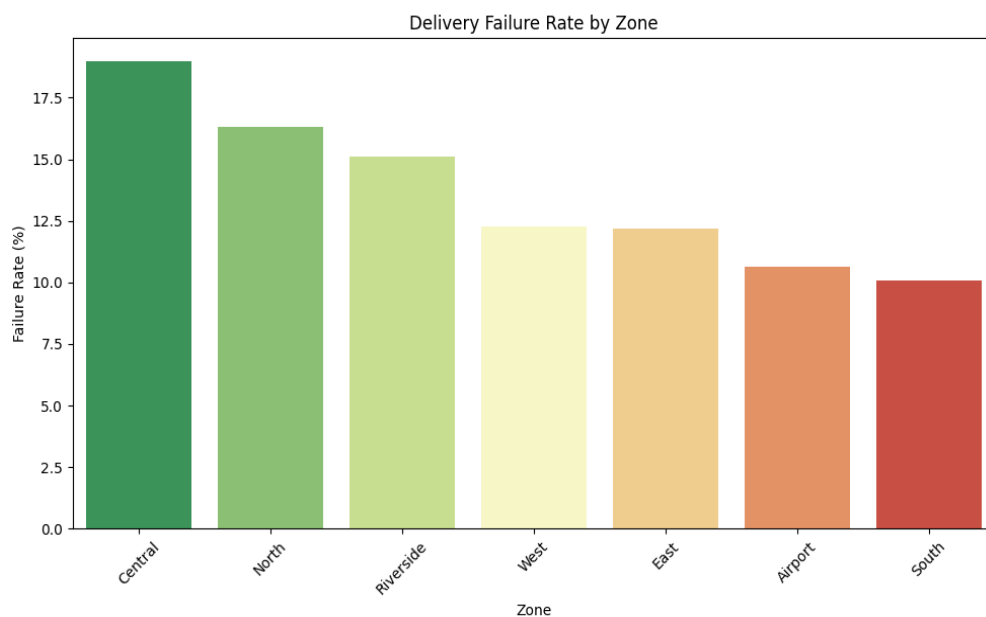


Figure 7: Delivery Failure Rate by Zone (Python)

The second plot is a correlation heatmap using the coolw arm colour scheme. The values match the R matrix: duration versus failed at 0.37 and override count versus failed at 0.02. Getting the same numbers from two separate implementations gives more confidence in the results.



Figure 8: Correlation Heatmap - Delivery Metrics (Python)

The third plot shows total compensation per complaint type as a bar chart. Delay leads at over GBP 1,680, followed by MissedPickup at around GBP 1,420. AppIssue and DriverBehaviour are both near GBP 980. This ranking gives the customer experience team a financially grounded list of what to tackle first.

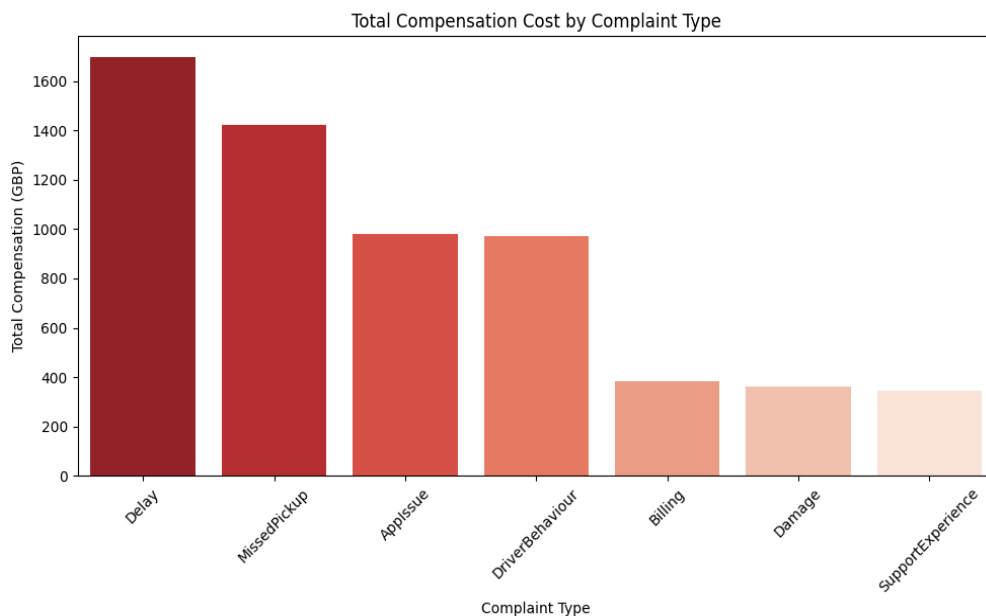


Figure 9: Total Compensation Cost by Complaint Type (Python)

The fourth plot looks at battery health versus incident count in Python and comes back with the same near-flat regression line as the R version. The result is consistent across tools.

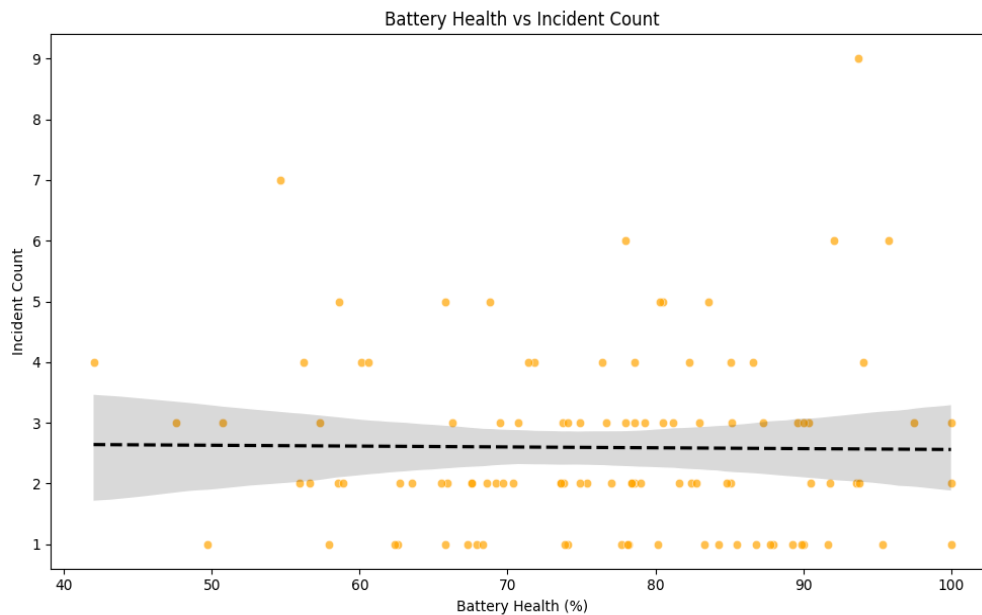


Figure 10: Battery Health vs Incident Count (Python)

The fifth plot ranks the top 15 drivers by failure rate, keeping only those with at least three deliveries. D063 leads at around 67%, with D092 at 60% and D104 at 57%. Those three are failing at roughly three to four times the network average and are the most obvious candidates for a performance review.

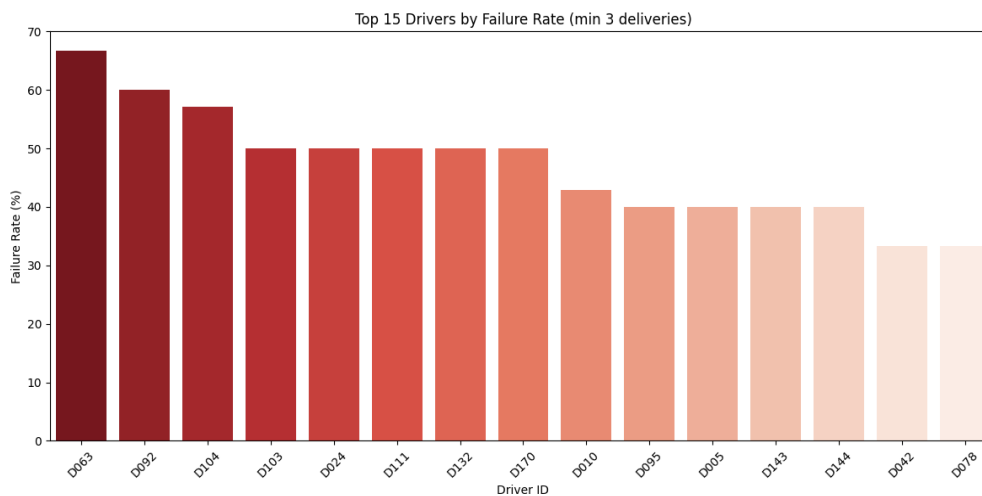


Figure 11: Top 15 Drivers by Failure Rate (Python)

The sixth plot shows app event failure rate by zone. Riverside tops it at 9.2%, yet its delivery failure rate sits in the middle of the pack. The app platform and the physical delivery operation are not failing in the same places, so they are likely driven by separate causes that need separate fixes.

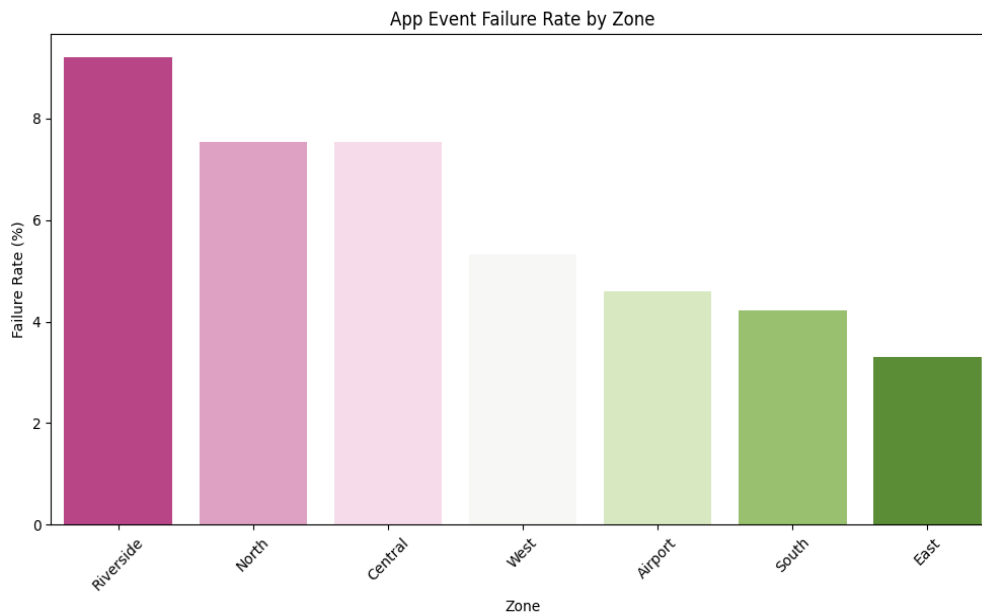


Figure 12: App Event Failure Rate by Zone (Python)

The seventh plot shows monthly order volume by service type from January 2024 to late 2025. Parcel and Passenger services carry the most volume throughout. No single service type shows a sustained climb, which suggests the cost increases seen across the business are not just down to one area growing quickly but are spread more broadly.

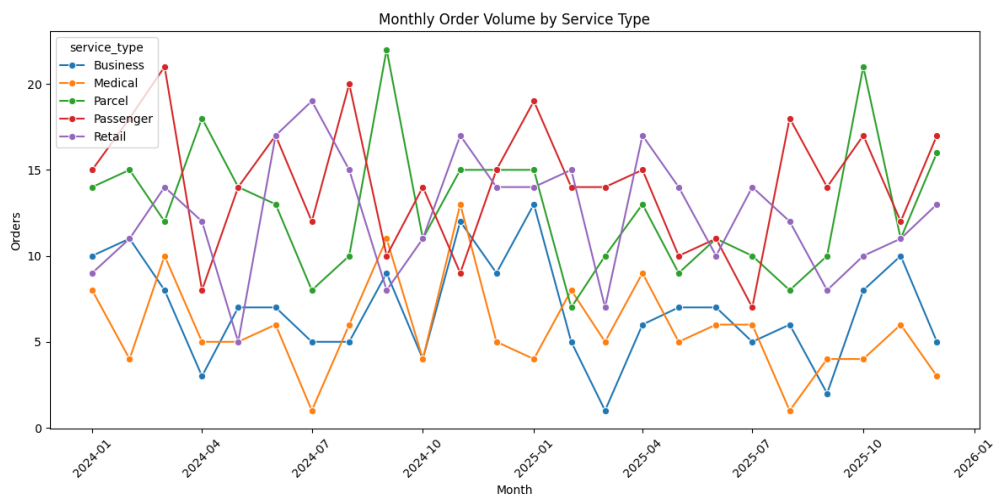


Figure 13: Monthly Order Volume by Service Type (Python)

MongoDB

Why NoSQL

The relational approach in Notebooks 01 and 02 works well for flat structured data but becomes awkward when records have nested histories or variable-length event logs. Complaint histories, driver incident records and app event streams all have that kind of irregular shape. Fitting them into a relational model would mean a lot of null-filled rows or complicated joins just to read back a single case. MongoDB Atlas lets related data live together in one document, so there are no joins needed at query time and the integrated customer view that management has been asking for becomes achievable.

Collection Design

Five collections were designed: `customer_profiles`, `deliveries`, `drivers`, `hubs` and `app_events`. Embedding decisions were driven by query patterns rather than normalisation, which is the correct approach for a document store.

Collection	What Was Embedded	Why
<code>customer_profiles</code>	Complaints array per customer	Complaints are always needed alongside customer data, so embedding removes the join that currently splits this across two systems
<code>deliveries</code>	Order information merged in	Order details are needed in nearly every delivery query, so pre-merging them removes join overhead at query time
<code>drivers</code>	Incidents array per driver	Incident history is always relevant when reviewing driver risk, so embedding makes pattern detection across events straightforward
<code>hubs</code>	Vehicles array per hub	Vehicle inventory is tied directly to hub capacity, so embedding supports vehicle count queries without a separate lookup
<code>app_events</code>	Nothing embedded	Events are independent with no linked history, so embedding would add complexity for no gain

Data Insertion

Data was inserted from normalised pandas dataframes after zone normalisation was re-applied. Pandas NaT values in datetime columns were replaced with None before converting rows to dictionaries, which avoids serialisation errors in PyMongo. Final collection counts were: 650 customer profiles, 950 deliveries, 170 drivers, 8 hubs and 640 app events.

CRUD Operations

For Create, all five collections were built using `insert_many()` on dictionaries from `to_dict('records')`. Complaints were grouped by `customer_id` before insertion so each customer document already contained its full complaints array on the way in.

For Read, three queries were written. The first retrieved failed deliveries in North zone, projecting only `delivery_id`, `driver_id` and `pickup_zone`. The second queried `customer_profiles` for documents containing at least one Open complaint, which would require a join in a relational model. The third identified high-risk drivers whose incidents array had three or more elements, using `$exists` on index position 2.

For Update, `update_one()` set `max_capacity` to 60 on hub H01. `update_many()` used `$elemMatch` to find customers with at least one High severity Open complaint and flagged them as `priority_customer: True`.

For Delete, `delete_many()` removed `app_events` documents where `zone_context` was None, clearing records that lacked geographic context and would have skewed zone-level aggregations.

Aggregation Pipelines

Four aggregation pipelines were built to draw analytical results directly from MongoDB.

Pipeline	What It Does	Main Result
Zone Failure Rate	Groups by pickup_zone, calculates total and failed counts, derives failure rate, sorts descending	Central zone highest at around 19%, consistent with SQL and Python
Driver Ranking	Groups by driver_id, averages customer rating, sums failures, limits to 10	Top 10 worst-performing drivers identified by total failures
Complaint Breakdown	Unwinds embedded complaints, groups by type and severity, sorts by count	Delay/Medium combination most frequent across all customers
Hub Performance	\$lookup join from hubs to deliveries by zone, counts totals and failures per hub	H05 in Central confirmed: 0 vehicles assigned, 11 failed deliveries

The \$lookup pipeline is particularly useful here as it joins hubs to deliveries inside MongoDB and independently surfaces the H05 resource gap, confirming the SQL Query 4 finding without touching the relational data.

Indexing and Query Performance

Five indexes were created on deliveries and one on customer_profiles:

- Single index on delivery_status
- Single index on pickup_zone
- Single index on driver_id
- Compound index on (pickup_zone, delivery_status) targeting the most common filter pattern
- Index on customer_profiles(complaints.status) for embedded array queries

A before and after timing test queried for failed deliveries in North zone and recorded elapsed milliseconds each time. The difference was modest given the 950-document dataset, as network latency to Atlas dominates at that scale. The explain plan from .explain() showed the winning plan as IXSCAN on the compound index rather than a COLLSCAN across all documents, confirming the indexes are being used correctly and will give more meaningful gains as the data grows.

Conclusion

The three notebooks produced a consistent set of findings across SQL, R, Python and MongoDB. The same problems kept surfacing regardless of which tool was used, which gives the conclusions more weight.

Central zone has the highest delivery failure rate at around 19% and hub H05 there has no vehicles assigned despite active delivery demand. That is the most concrete and fixable problem identified. Failed deliveries take significantly longer than successful ones, confirmed statistically at $p = 8.61e-26$, and duration is the single strongest predictor of failure in the dataset. Route overrides, by contrast, show virtually no relationship with failure at 0.02 correlation across both R and Python, so they are not the cause of poor outcomes despite being a common assumption. Delay is both the most frequent complaint type and the most

expensive at over GBP 1,680 in compensation. Medium severity complaints have remained persistently high since 2024 with no sign of improvement. Battery health is a poor standalone predictor of incidents. Driver D063 has a 67% failure rate, nearly four times the network average. And the mobile app fails most in Riverside zone even though deliveries there are not particularly problematic, pointing to separate infrastructure issues.

The most urgent action is reallocating vehicles to Central zone and investigating why H05 is running without any fleet. After that, reducing delivery delays should be the operational priority given how directly duration ties to both failures and compensation costs. The MongoDB `customer_profiles` structure should be taken towards production to replace the fragmented customer view that currently exists. Drivers D063, D092 and D104 need a performance review. Maintenance scheduling needs to incorporate more than just battery level. The override investigation should be dropped as the data does not support it. And app reliability in Riverside needs looking at separately from the delivery operation.

References

- NorthStar Case Study. (2026). Databases and Analytics Module. University of West London.
- Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York.
- McKinney, W. (2017). *Python for Data Analysis*. 2nd ed. O'Reilly Media.
- MongoDB Inc. (2024). MongoDB Documentation. Available at: <https://www.mongodb.com/docs/>
- Virtanen, P. et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, pp.261-272.
- R Core Team. (2023). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna.